

Akshita Bhamidimarri

Capstone Project - 1

IOT-Enabled Retail Analytics on Azure Databricks

1) Creating a storage account

The screenshot shows the Microsoft Azure portal interface. The browser tabs include 'de-with-azure-banglore', 'Capstone Project 1 - 5 Hoi...', 'Capstone-1 - Google Docs', 'aksssstore - Microsoft Azure', and 'Create a virtual machine'. The URL is 'https://portal.azure.com/#@michaelkloudkraft@hotmail.onmicrosoft.com/resource/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resour...'. The page title is 'aksssstore | Overview'. The left sidebar shows the 'Overview' tab selected. The main content area displays the 'Essentials' section with the following details:

- Resource group: [LabsKraft2027](#)
- Location: eastus
- Subscription: [LabsKraft](#)
- Subscription ID: d008c7cc-9795-4292-bf79-74ae52cda63d
- Disk state: Available
- Tags: [Add tags](#)
- Performance: Standard
- Replication: Locally-redundant storage (LRS)
- Account kind: StorageV2 (general purpose v2)
- Provisioning state: Succeeded
- Created: 8/25/2025, 10:03:00 AM

Below the Essentials section, there are tabs for 'Properties', 'Monitoring', 'Capabilities (5)', 'Recommendations (0)', 'Tutorials', and 'Tools + SDKs'. The 'Properties' tab is active, showing 'Data Lake Storage' and 'Security' sections. The 'Data Lake Storage' section shows 'Hierarchical namespace' as 'Enabled' and 'Default access tier' as 'Hot'. The 'Security' section shows 'Require secure transfer for REST API operations' as 'Enabled'.

2) Creating a container inside the storage account

The screenshot shows the Microsoft Azure portal interface. The browser tabs include 'de-with-azure-banglore', 'Capstone Project 1 - 5 Hoi...', 'Capstone-1 - Google Docs', 'aksssstore - Microsoft Azure', 'akssconnect - Microsoft Azure', 'Catalog Explorer', and 'Catalog Explorer'. The URL is 'https://portal.azure.com/#@michaelkloudkraft@hotmail.onmicrosoft.com/resource/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/r...'. The page title is 'aksssstore | Containers'. The left sidebar shows the 'Containers' tab selected. The main content area displays the 'Containers' section with the following details:

- Search containers by prefix:
- Only show active containers:
- Showing all 2 items
- Table with 4 columns: Name, Last modified, Anonymous access level, Lease state

Name	Last modified	Anonymous access level	Lease state
\$logs	8/25/2025, 10:03:25 AM	Private	Available
akssscont	8/25/2025, 10:33:21 AM	Private	Available

3) Adding the csv file inside the storage account

The screenshot shows the Microsoft Azure portal interface. The breadcrumb navigation indicates the path: Home > aksssstore | Containers > akssscont. The container 'akssscont' is selected, and the 'Overview' tab is active. The authentication method is 'Access key'. A search bar for blobs is present. A table lists the contents of the container, showing one item: 'retail_iot_data.csv'.

Name	Last modified	Access tier	Blob type	Size	Lease state
retail_iot_data.csv	8/25/2025, 10:34:00 AM	Hot (Inferred)	Block blob	130.76 KiB	Available

4) Creating an access connector for databricks

The screenshot shows the Microsoft Azure portal interface for an 'akconnect' Access Connector for Azure Databricks. The breadcrumb navigation indicates the path: Home > AccessConnectorCreate-20250825102606 | Overview > LabsKraft2027 >. The 'Overview' tab is active. The connector is in a 'Succeeded' state. Essential information is displayed, including the resource group, location, subscription ID, and resource ID.

Essentials	JSON View
Resource group (move) LabsKraft2027	State Succeeded
Location East US	Resource ID /subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/...
Subscription (move) LabsKraft	
Subscription ID d008c7cc-9795-4292-bf79-74ae52cda63d	
Tags (edit) Add tags	

5) Creating the external location for unity catalog

The image consists of two screenshots of the Databricks web interface, showing the process of creating and testing an external location.

Top Screenshot: The 'Catalog Explorer' view for 'External Locations' shows a new location named 'aks_proj1'. The 'Overview' tab is selected. The 'Description' section has an 'Add description' button. The 'About this external location' section shows the owner as 'Tredence LabsKraft2' and the fallback mode as 'Disabled'. A table lists the configuration details:

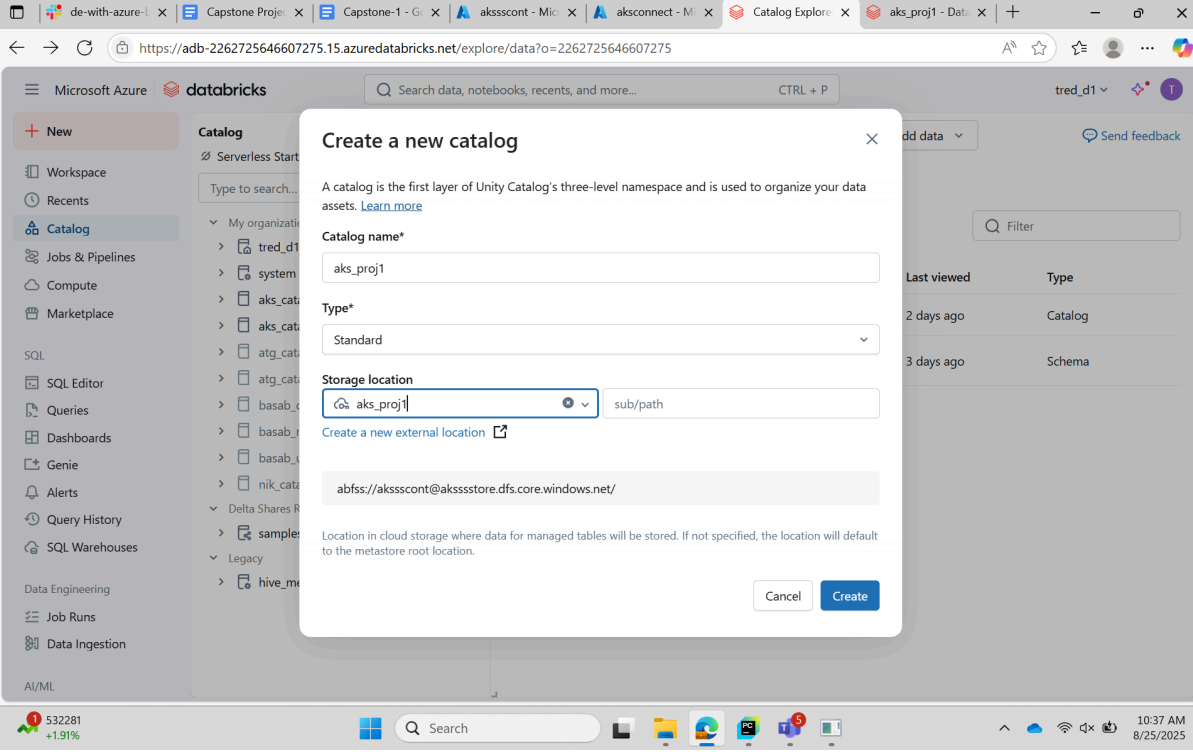
Property	Value
Credential	aks_proj1_azuremanagedidentity_1756098357660
URL	abfss://aksscont@akssstore.dfs.core.windows.net/
Limit to read-only use	Disabled

Bottom Screenshot: The same 'aks_proj1' location is shown, but a 'Test connection' dialog box is open. The dialog displays the following information:

- Location Type:** Directory
- Permissions:**
 - Success - Read
 - Success - List
 - Success - Write
 - Success - Delete
 - Success - Path Exists
 - Success - Hierarchical Namespace Enabled
- All Permissions Confirmed:** The associated Storage Credential grants permission to perform all necessary operations.

The 'Done' button is visible at the bottom of the dialog box.

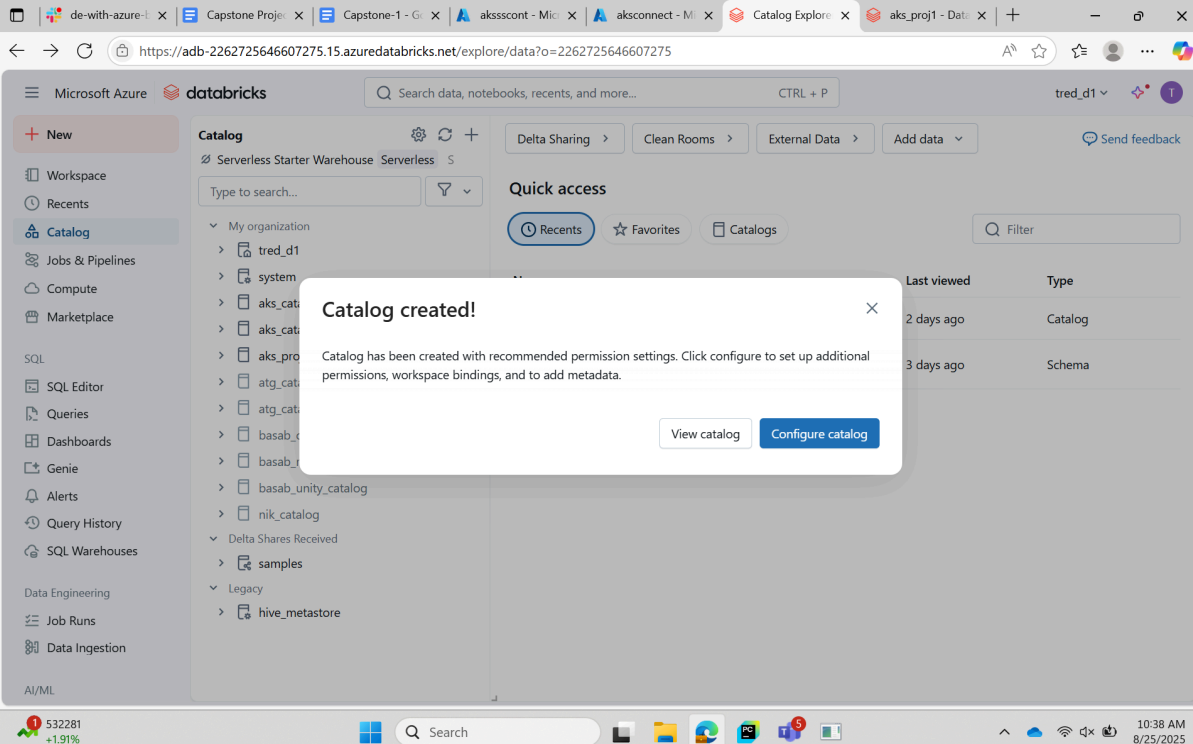
6) Creating the unity catalog using the previously created external location



The screenshot shows the Databricks interface with the 'Create a new catalog' dialog box open. The dialog contains the following fields and options:

- Catalog name***: aks_proj1
- Type***: Standard (dropdown menu)
- Storage location**: abfss://aksscont@akssstore.dfs.core.windows.net/ (with a 'Create a new external location' link)

Below the storage location, there is a note: "Location in cloud storage where data for managed tables will be stored. If not specified, the location will default to the metastore root location." The dialog has 'Cancel' and 'Create' buttons.



The screenshot shows the Databricks interface after the catalog has been created. A 'Catalog created!' dialog box is displayed with the following text:

Catalog has been created with recommended permission settings. Click configure to set up additional permissions, workspace bindings, and to add metadata.

Buttons: View catalog, Configure catalog

7) Deploying a new azure synapse analytics

Home > Microsoft.Azure.SynapseAnalytics-20250825110509 | Overview

Search resources, services, and docs (G+/)

Overview

Inputs

Outputs

Template

✓ Your deployment is complete

Deployment name : Microsoft.Azure.SynapseAnaly... Start time : 8/25/2025, 11:07:07 AM
Subscription : LabsKraft Correlation ID : 728ed17e-8f16-4fd9-b8b8-bd...
Resource group : LabsKraft2027

Deployment details

Next steps

Go to resource group

Give feedback

Tell us about your experience with deployment

Cost management

Get notified to stay within your budget and prevent unexpected charges on your bill.
[Set up cost alerts >](#)

Microsoft Defender for Cloud

Secure your apps and infrastructure
[Go to Microsoft Defender for Cloud >](#)

Free Microsoft tutorials

[Start learning today >](#)

Work with an expert

Azure experts are service provider partners who can help manage your assets on Azure and be your first line of support.
[Find an Azure expert >](#)

Home > Microsoft.Azure.SynapseAnalytics-20250825110509 | Overview > LabsKraft2027

aksssynapse Synapse workspace

Search

New dedicated SQL pool New Apache Spark pool Refresh Reset SQL admin password Delete

Overview

Activity log

Access control (IAM)

Tags

Diagnose and solve problems

Resource visualizer

Settings

Analytics pools

Security

Monitoring

Automation

Help

Essentials

Resource group (move)
[LabsKraft2027](#)

Status
Succeeded

Location
East US

Subscription (move)
[LabsKraft](#)

Subscription ID
d008c7cc-9795-4292-bf79-74ae52cda63d

Managed virtual network
No

Managed Identity object ID
8a3e525c-c8dd-42e8-8a26-b79eeab446d1

Workspace web URL
<https://web.azuresynapse.net?workspace=%2fsubscriptions%2fd008c7cc-...>

Tags (edit)
[Add tags](#)

Networking
[Show firewall settings](#)

Primary ADLS Gen2 account URL
<https://akssstore.dfs.core.windows.net>

Primary ADLS Gen2 file system
aksscont

SQL admin username
sqladminuser

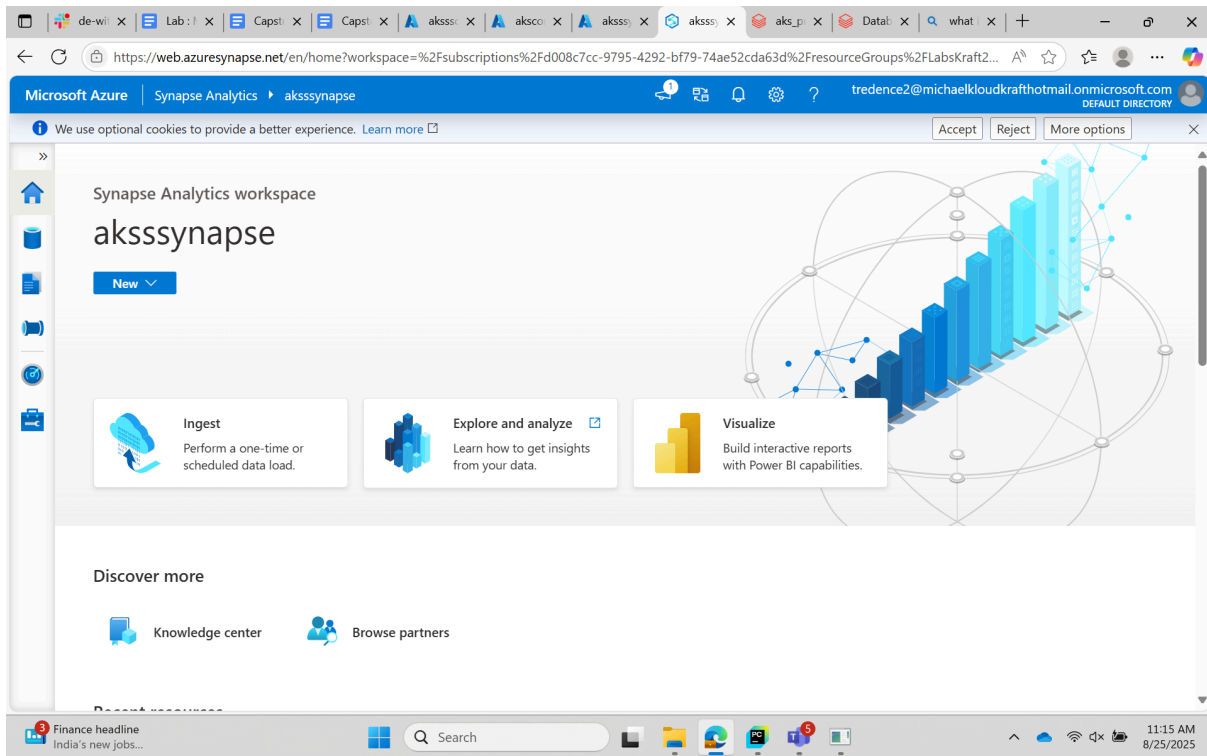
SQL Microsoft Entra admin
tredence2@michaelkloudkraft@hotmail.onmicrosoft.com

Dedicated SQL endpoint
aksssynapse.sql.azuresynapse.net

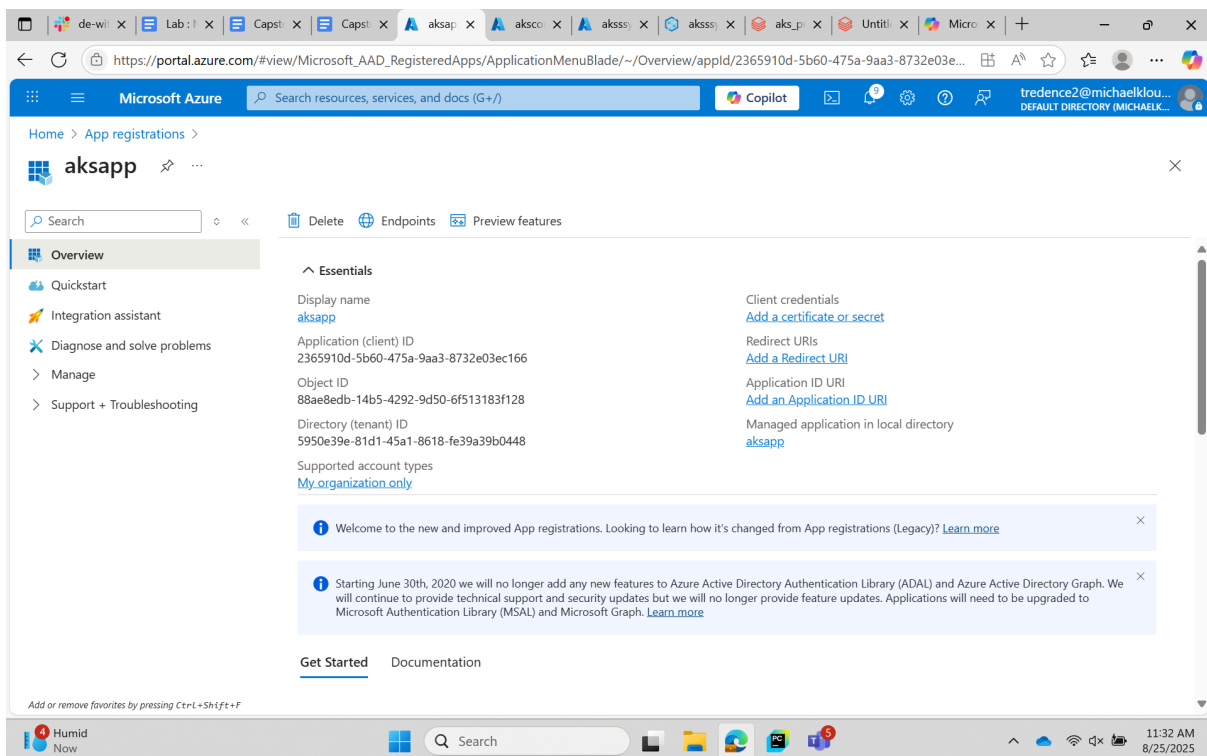
Serverless SQL endpoint
aksssynapse-ondemand.sql.azuresynapse.net

Development endpoint
<https://aksssynapse.dev.azuresynapse.net>

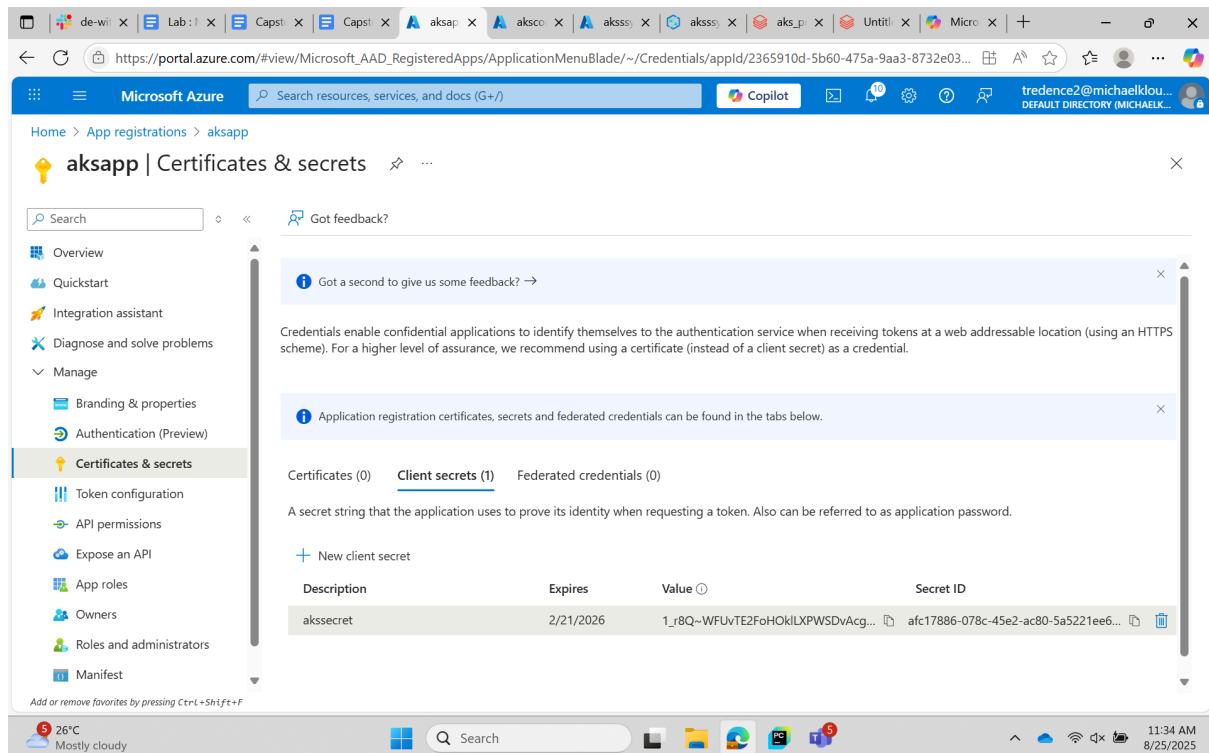
Getting started



8) Creating a new app registration



9) Generating a new secret value

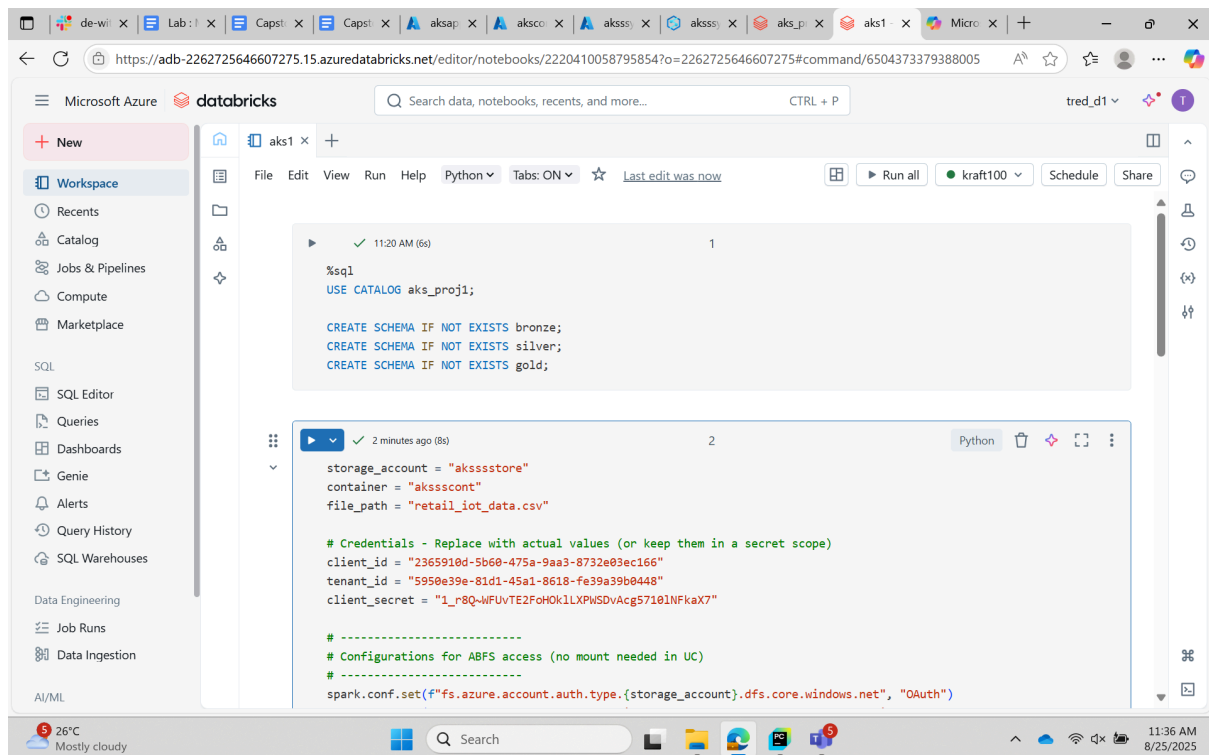


10) Creating three schemas in the medallion architecture

Code for schema creation -

```
%sql
USE CATALOG aks_proj1;

CREATE SCHEMA IF NOT EXISTS bronze;
CREATE SCHEMA IF NOT EXISTS silver;
CREATE SCHEMA IF NOT EXISTS gold;
```



Code for bronze layer:

```
storage_account = "aksssstore"
container = "akssscont"
file_path = "retail_iot_data.csv"

# Credentials - Replace with actual values (or keep them in a secret scope)
client_id = "2365910d-5b60-475a-9aa3-8732e03ec166"
tenant_id = "5950e39e-81d1-45a1-8618-fe39a39b0448"
client_secret = "1_r8Q~WFUvTE2FoHOk1LXPWSDvAcg57101NFkaX7"

# -----
# Configurations for ABFS access (no mount needed in UC)
# -----

spark.conf.set(f"fs.azure.account.auth.type.{storage_account}.dfs.core.window
s.net", "OAuth")

spark.conf.set(f"fs.azure.account.oauth.provider.type.{storage_account}.dfs.c
ore.windows.net",

"org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider")

spark.conf.set(f"fs.azure.account.oauth2.client.id.{storage_account}.dfs.core
.windows.net", client_id)
```



```

spark.conf.set(f"fs.azure.account.oauth2.client.secret.{storage_account}.dfs.
core.windows.net", client_secret)
spark.conf.set(f"fs.azure.account.oauth2.client.endpoint.{storage_account}.df
s.core.windows.net",
               f"https://login.microsoftonline.com/{tenant_id}/oauth2/token")

# -----
# Build ABFS path directly (no /mnt needed)
# -----
abfs_path =
f"abfss://{container}@{storage_account}.dfs.core.windows.net/{file_path}"

# -----
# Try reading the file to verify access
# -----
# Read raw file from ADLS
df_bronze = (
    spark.read
        .option("header", "true")
        .option("inferSchema", "true")    # or provide schema for better
performance
        .csv(abfs_path)
)

print(f"✅ Loaded {df_bronze.count()} rows from {abfs_path}")
display(df_bronze.limit(5))

```

```

catalog = "aks_proj1"
schema = "bronze"
table_name = "bronze"

# Write into UC Bronze Delta table
df_bronze.write.format("delta").mode("overwrite").saveAsTable(
    f"{catalog}.{schema}.{table_name}"
)

print(f"✅ Bronze Delta table created: {catalog}.{schema}.{table_name}")

```

Microsoft Azure databricks Search data, notebooks, recents, and more... CTRL + P tred_d1

New Workspace Recents Catalog Jobs & Pipelines Compute Marketplace SQL SQL Editor Queries Dashboards Genie Alerts Query History SQL Warehouses Data Engineering Job Runs Data Ingestion AI/ML

aks1 x + File Edit View Run Help Python Tabs: ON Last edit was now Run all kraft100 Schedule Share

2 Python

```
spark.read
  .option("header", "true")
  .option("inferSchema", "true") # or provide schema for better performance
  .csv(abfs_path)

print(f"Loaded {df_bronze.count()} rows from {abfs_path}")
display(df_bronze.limit(5))
```

(9) Spark Jobs

df_bronze: pyspark.sql.connect.dataframe.DataFrame = [OrderID: string, CustomerID: string ... 12 more fields]

Loaded 1000 rows from abfss://aksscont@akssstore.dfs.core.windows.net/retail_iot_data.csv

	OrderID	CustomerID	Region	StoreID	Product	Quantity
1	db07ec0d-e8e7-408a-846e-c2d330484d...	CUST-2824	East	Store-009	Mobile	2
2	93d89258-6ef2-4bb8-9a71-cb590cb55f85	CUST-7912	East	Store-001	Laptop	2
3	f97636e9-3dab-48ad-aad4-0eca034d3111	CUST-9928	South	Store-008	Tablet	5
4	f20eae76-3e09-4580-8429-af1d6b76209a	CUST-3547	West	Store-011	Laptop	1
5	...	...	...	...	...	...

5 rows | 8.08s runtime Refreshed 2 minutes ago

26°C Mostly cloudy 11:37 AM 8/25/2025

Microsoft Azure databricks Search data, notebooks, recents, and more... CTRL + P tred_d1

New Workspace Recents Catalog Jobs & Pipelines Compute Marketplace SQL SQL Editor Queries Dashboards Genie Alerts Query History SQL Warehouses Data Engineering Job Runs Data Ingestion AI/ML

aks1 x + File Edit View Run Help Python Tabs: ON Last edit was now Run all kraft100 Schedule Share

5 Python

5 rows | 8.08s runtime Refreshed 6 minutes ago

```
catalog = "aks_proj1"
schema = "bronze"
table_name = "bronze"

# Write into UC Bronze Delta table
df_bronze.write.format("delta").mode("overwrite").saveAsTable(
  f"{catalog}.{schema}.{table_name}"
)

print(f"Bronze Delta table created: {catalog}.{schema}.{table_name}")
```

(1) Spark Jobs

Bronze Delta table created: aks_proj1.bronze.bronze

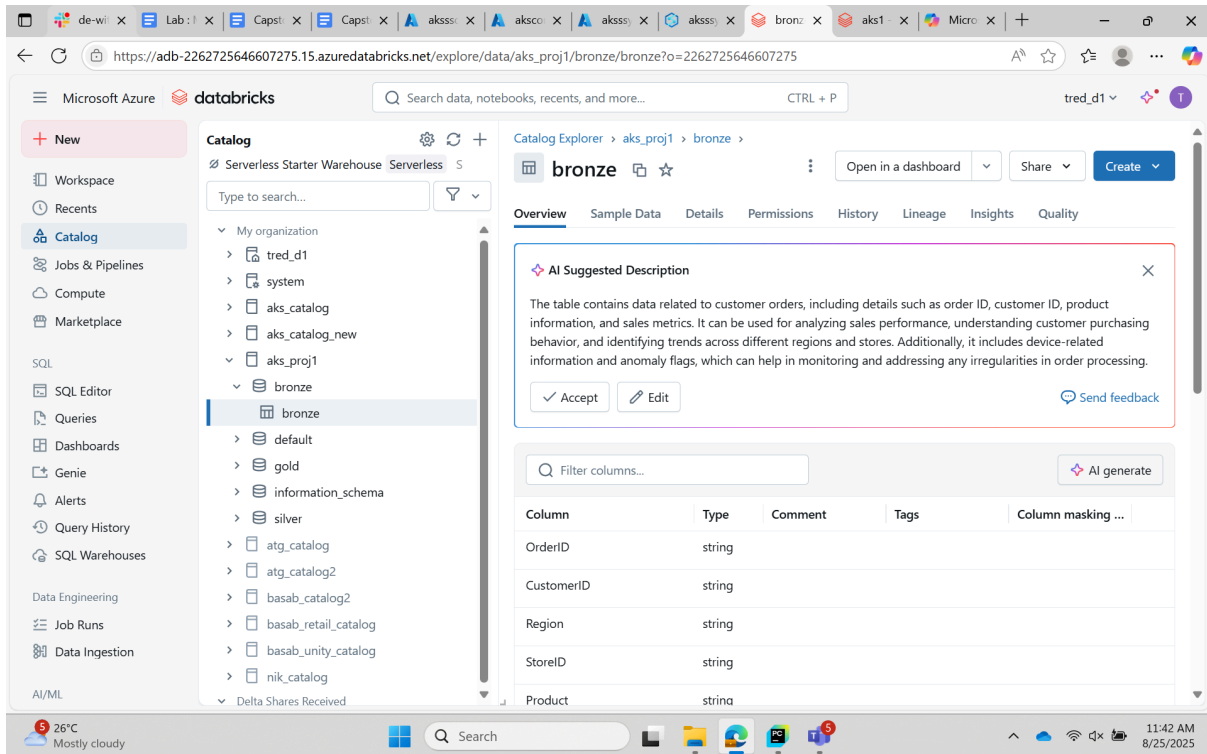
4 Python

Start typing or generate with AI (Ctrl + I)...

+ Code + Text Assistant

26°C Mostly cloudy 11:41 AM 8/25/2025

11) Bronze table that we just created in the schema with the appropriate columns



Microsoft Azure databricks

Search data, notebooks, recents, and more... CTRL + P

tred_d1

New

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

AI/ML

Delta Shares Received

Catalog Explorer > aks_proj1 > bronze >

bronze

Open in a dashboard

Share

Create

Overview Sample Data Details Permissions History Lineage Insights Quality

AI Suggested Description

The table contains data related to customer orders, including details such as order ID, customer ID, product information, and sales metrics. It can be used for analyzing sales performance, understanding customer purchasing behavior, and identifying trends across different regions and stores. Additionally, it includes device-related information and anomaly flags, which can help in monitoring and addressing any irregularities in order processing.

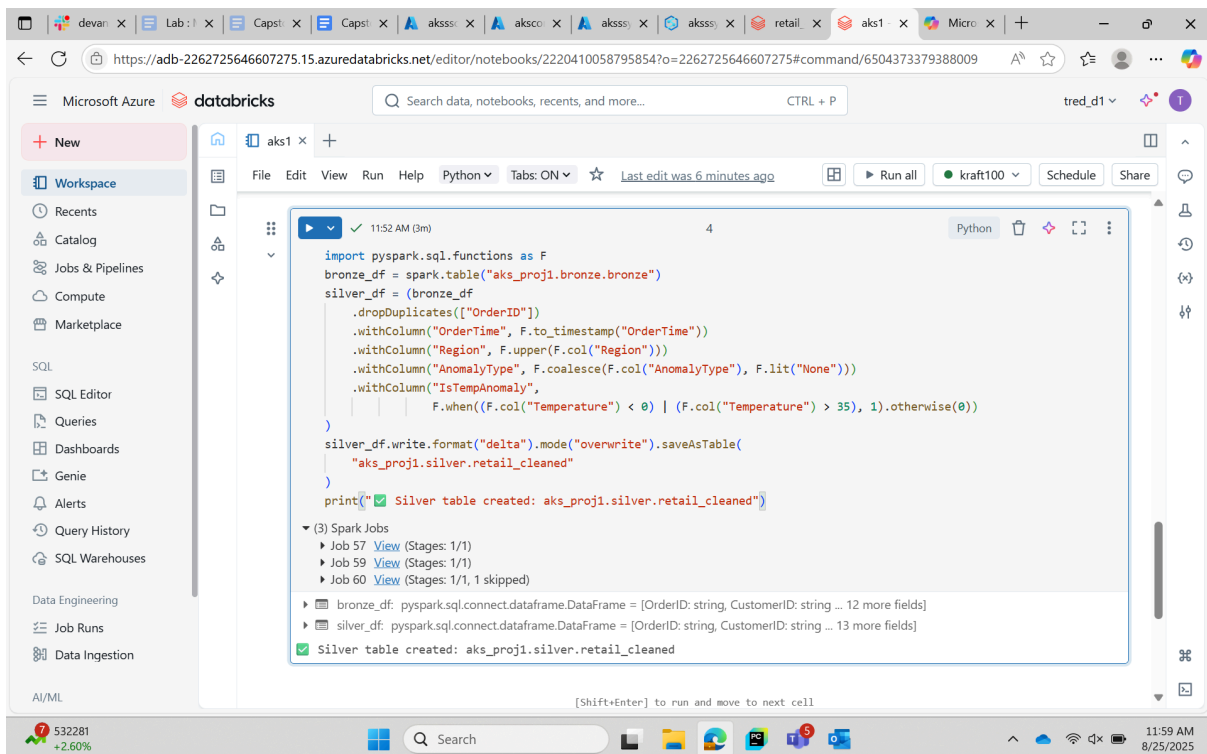
Accept Edit Send feedback

Filter columns...

AI generate

Column	Type	Comment	Tags	Column masking ...
OrderID	string			
CustomerID	string			
Region	string			
StoreID	string			
Product	string			

12) Creating the silver layer



Microsoft Azure databricks

Search data, notebooks, recents, and more... CTRL + P

tred_d1

New

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

AI/ML

aks1

File Edit View Run Help Python Tabs: ON

Last edit was 6 minutes ago

Run all

kraft100

Schedule

Share

```
import pyspark.sql.functions as F
bronze_df = spark.table("aks_proj1.bronze.bronze")
silver_df = (bronze_df
    .dropDuplicates(["OrderID"])
    .withColumn("OrderTime", F.to_timestamp("OrderTime"))
    .withColumn("Region", F.upper(F.col("Region")))
    .withColumn("AnomalyType", F.coalesce(F.col("AnomalyType"), F.lit("None")))
    .withColumn("IsTempAnomaly",
        F.when((F.col("Temperature") < 0) | (F.col("Temperature") > 35), 1).otherwise(0))
    )
silver_df.write.format("delta").mode("overwrite").saveAsTable(
    "aks_proj1.silver.retail_cleaned"
)
print("Silver table created: aks_proj1.silver.retail_cleaned")
```

(3) Spark Jobs

- Job 57 View (Stages: 1/1)
- Job 59 View (Stages: 1/1)
- Job 60 View (Stages: 1/1, 1 skipped)

bronze_df: pyspark.sql.connect.dataframe.DataFrame = [OrderID: string, CustomerID: string ... 12 more fields]

silver_df: pyspark.sql.connect.dataframe.DataFrame = [OrderID: string, CustomerID: string ... 13 more fields]

Silver table created: aks_proj1.silver.retail_cleaned

Code for silver layer:

```
import pyspark.sql.functions as F
bronze_df = spark.table("aks_proj1.bronze.bronze")
silver_df = (bronze_df
    .dropDuplicates(["OrderID"])
    .withColumn("OrderTime", F.to_timestamp("OrderTime"))
    .withColumn("Region", F.upper(F.col("Region")))
    .withColumn("AnomalyType", F.coalesce(F.col("AnomalyType"),
F.lit("None")))
    .withColumn("IsTempAnomaly",
        F.when((F.col("Temperature") < 0) | (F.col("Temperature") >
35), 1).otherwise(0))
)
silver_df.write.format("delta").mode("overwrite").saveAsTable(
    "aks_proj1.silver.retail_cleaned"
)
print("✅ Silver table created: aks_proj1.silver.retail_cleaned")
```

Microsoft Azure databricks

Search data, notebooks, recents, and more... CTRL + P

treed_d1

New

- Workspace
- Recents
- Catalog
- Jobs & Pipelines
- Compute
- Marketplace
- SQL
- SQL Editor
- Queries
- Dashboards
- Genie
- Alerts
- Query History
- SQL Warehouses
- Data Engineering
- Job Runs
- Data Ingestion
- AI/ML

Catalog

Serverless Starter Warehouse Serverless S

Type to search...

- system
- aks_catalog
- aks_catalog_new
- aks_proj1
 - bronze
 - bronze
 - default
 - gold
 - information_schema
 - silver
 - retail_cleaned
- atg_catalog
- atg_catalog2
- basab_catalog2
- basab_retail_catalog
- basab_unity_catalog
- nik_catalog
- Delta Shares Received
- camnloc

Catalog Explorer > aks_proj1 > silver

retail_cleaned

Open in a dashboard

Share

Create

Overview Sample Data Details Permissions History Lineage Insights Quality

AI Suggested Description

The table contains data related to customer orders, including details such as order identification, customer information, product specifics, and sales metrics. It also records the time of the order and the device used for the transaction, along with environmental factors like temperature. This data can be utilized for analyzing sales trends, customer purchasing behavior, and identifying any anomalies in order processing or environmental conditions.

Accept Edit

Send feedback

Filter columns...

AI generate

Column	Type	Comment	Tags	Column masking ...
OrderID	string			
CustomerID	string			
Region	string			
StoreID	string			
Product	string			

532281 +2.60%

12:00 PM 8/25/2025

13) Creating different KPI's for the gold layer

Code for KPI 1 (daily/monthly sales per region):

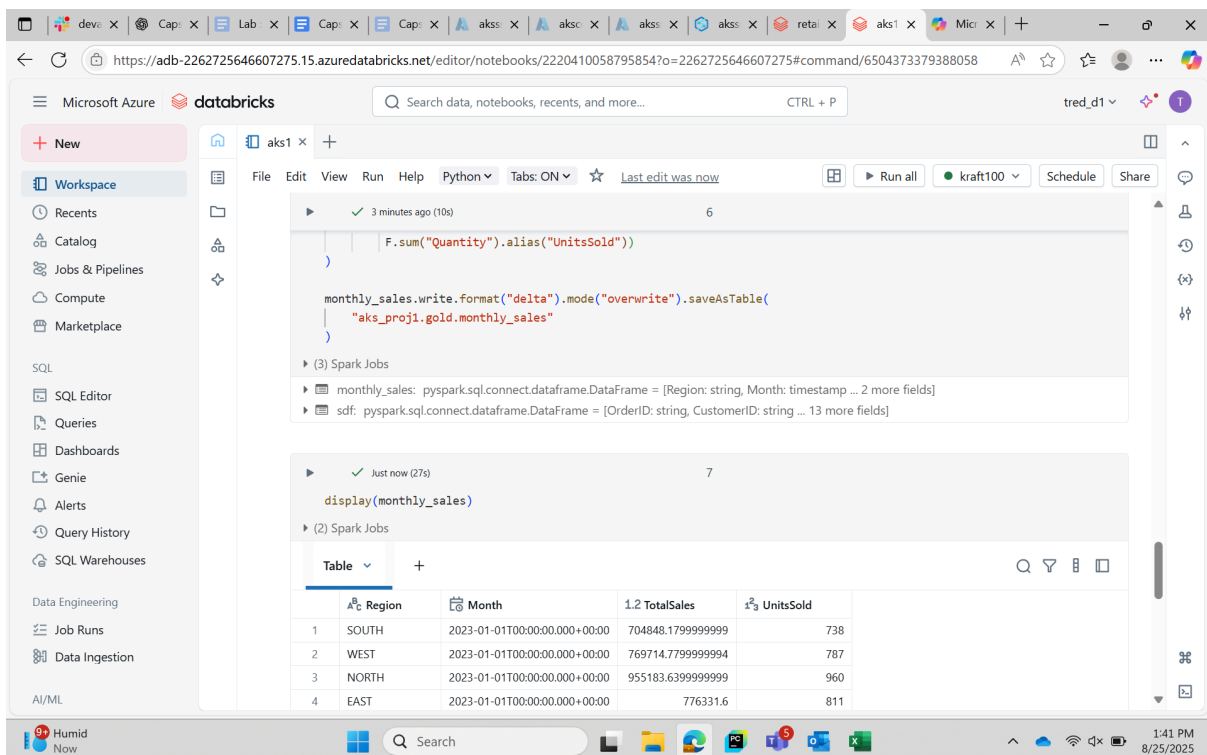
```
#gold layer kpi1 - dialy/monthly sales per region

import pyspark.sql.functions as F

sdf = spark.table("aks_proj1.silver.retail_cleaned")

monthly_sales = (sdf
    .groupBy("Region", F.date_trunc("month", "OrderTime").alias("Month"))
    .agg(F.sum("SalesAmount").alias("TotalSales"),
        F.sum("Quantity").alias("UnitsSold"))
)

monthly_sales.write.format("delta").mode("overwrite").saveAsTable(
    "aks_proj1.gold.monthly_sales"
)
```



The screenshot shows the Databricks workspace interface. The left sidebar contains navigation options like Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie, Alerts, Query History, SQL Warehouses, Data Engineering, Job Runs, Data Ingestion, and AI/ML. The main area displays a Python script being executed. The script defines a DataFrame `sdf` from the `aks\_proj1.silver.retail\_cleaned` table, groups it by `Region` and `Month` (using `date\_trunc` on `OrderTime`), and aggregates `SalesAmount` as `TotalSales` and `Quantity` as `UnitsSold`. The result is saved as a table named `aks\_proj1.gold.monthly\_sales`. Below the script, the execution results are shown, including a table with 4 rows and 4 columns: Region, Month, TotalSales, and UnitsSold.

	Region	Month	TotalSales	UnitsSold
1	SOUTH	2023-01-01T00:00:00.000+00:00	704848.1799999999	738
2	WEST	2023-01-01T00:00:00.000+00:00	769714.7799999994	787
3	NORTH	2023-01-01T00:00:00.000+00:00	955183.6399999999	960
4	EAST	2023-01-01T00:00:00.000+00:00	776331.6	811

Code for KPI 2 (anomaly trend per store):

```
#gold layer kpi2 - anomaly trend per store

anomaly_trend = (sdf
    .groupBy("StoreID", F.to_date("OrderTime").alias("Day"))
    .agg(F.sum("AnomalyFlag").alias("SystemAnomalies"),
        F.sum("IsTempAnomaly").alias("TempAnomalies"),
        F.sum("SalesAmount").alias("Sales"))
)

anomaly_trend.write.format("delta").mode("overwrite").saveAsTable(
    "aks_proj1.gold.anomaly_trend"
)
```

Code for KPI 3 (impact of device failure):

```
#gold layer kpi3 - impact of device failures

import pyspark.sql.functions as F
sdf = spark.table("aks_proj1.silver.retail_cleaned")

impact = (sdf
    .withColumn("AnomalyDetected", (F.col("AnomalyFlag") +
    F.col("IsTempAnomaly")) > 0)
    .groupBy("AnomalyDetected")
    .agg(F.sum("SalesAmount").alias("TotalSales"),
        F.countDistinct("OrderID").alias("Orders"))
)

impact.write.format("delta").mode("overwrite").saveAsTable(
    "aks_proj1.gold.anomaly_impact"
)
```

Code for KPI 4 (stores into tier):

```
#gold layer kpi4 - store into tiers

store_perf = (sdf
  .groupBy("StoreID")
  .agg(F.sum("SalesAmount").alias("TotalSales"))
  .withColumn("Tier",
    F.when(F.col("TotalSales") > 50000, "High")
    .when(F.col("TotalSales") > 20000, "Medium")
    .otherwise("Low"))
)

store_perf.write.format("delta").mode("overwrite").saveAsTable(
  "aks_proj1.gold.store_performance"
)
```

Code for KPI 5 (weighted average of sales vs anomalies):

```
#gold layer kpi5 - weighted average of sales vs anomalies

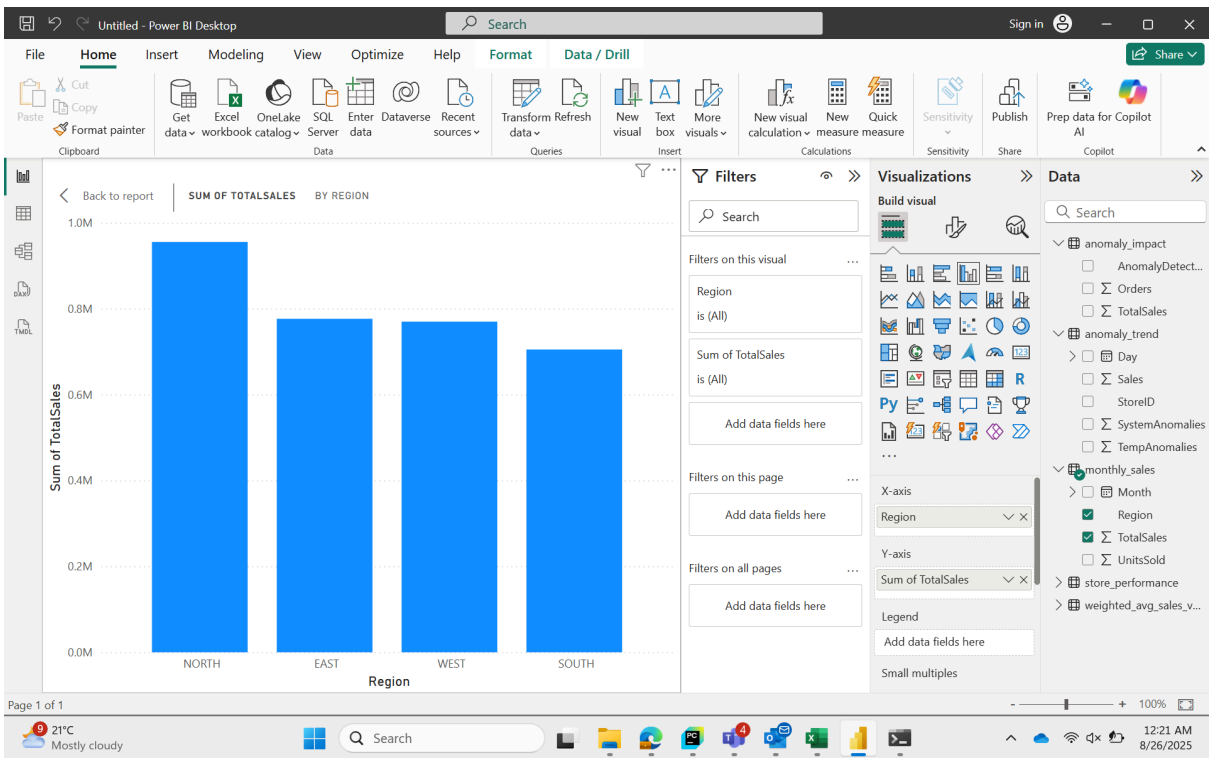
store_sales_anoms = (sdf
  .groupBy("StoreID")
  .agg(F.sum("SalesAmount").alias("TotalSales"),
    F.sum("AnomalyFlag").alias("AnomalyCount"))
  .filter("AnomalyCount > 0") # only consider stores with anomalies
)

# Compute weighted average: (Sales * AnomalyCount) / Sum(AnomalyCount)
weighted_avg = (store_sales_anoms
  .select((F.sum(F.col("TotalSales")) * F.col("AnomalyCount")) /
    F.sum("AnomalyCount")).alias("WeightedAvgSales"))
)

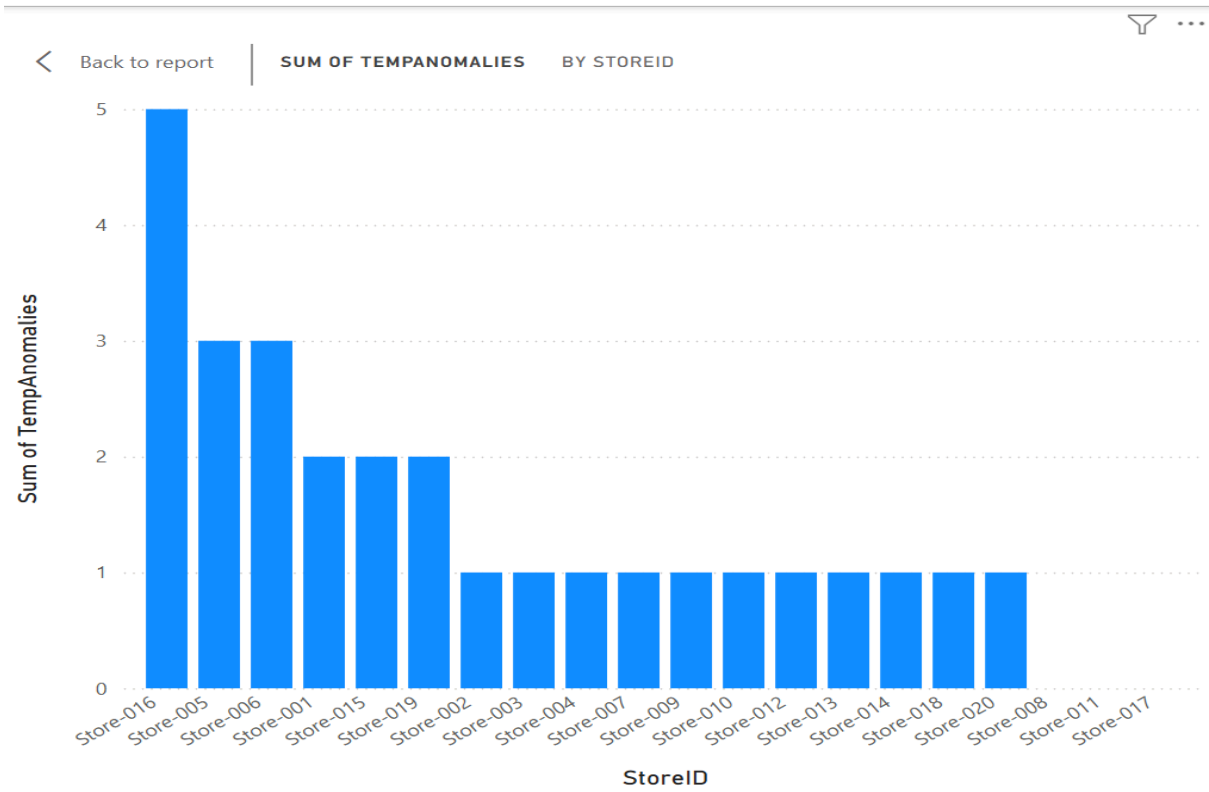
weighted_avg.write.format("delta").mode("overwrite").saveAsTable(
  "aks_proj1.gold.weighted_avg_sales_vs_anomalies"
)
```

14) Power BI dashboards

Total sales per region



Sum of Anomalies per store ID



15) Optimization of the queries

```
%sql
OPTIMIZE aks_proj1.gold.monthly_sales ZORDER BY (Month);

%sql
OPTIMIZE aks_proj1.gold.anomaly_trend ZORDER BY (StoreID, Day);

%sql
OPTIMIZE aks_proj1.gold.anomaly_impact ZORDER BY (AnomalyDetected);

%sql
OPTIMIZE aks_proj1.gold.store_performance ZORDER BY (TotalSales);

%sql
OPTIMIZE aks_proj1.gold.weighted_avg_sales_vs_anomalies;

%sql
OPTIMIZE aks_proj1.gold.monthly_sales;
OPTIMIZE aks_proj1.gold.anomaly_trend;
OPTIMIZE aks_proj1.gold.anomaly_impact;
OPTIMIZE aks_proj1.gold.store_performance;
OPTIMIZE aks_proj1.gold.weighted_avg_sales_vs_anomalies;
```

16) Using Event Hub as a Data Source

The screenshot displays the Microsoft Azure portal interface for an Event Hubs Namespace named 'aks-eventhub'. The left sidebar contains a navigation menu with options like Overview, Activity log, Access control (IAM), Tags, Diagnose and solve problems, Data Explorer, Resource visualizer, Events, Settings, Entities, Monitoring, Automation, and Help. The main content area is divided into two sections: 'Essentials' and 'JSON View'. The 'Essentials' section provides key information about the namespace, including its resource group ('LabsKraft2027'), status ('Succeeded'), location ('East US'), subscription ID, and host name. It also lists various configuration details such as pricing tier ('Basic'), throughput units ('1 unit'), and local authentication ('Enabled'). At the bottom, there are status indicators for 'NAMESPACE CONTENTS' (0 event hubs), 'KAFKA SURFACE' (not supported), and 'ZONE REDUNDANCY' (enabled). The bottom of the screen shows a Windows taskbar with the date and time as 8:29 PM on 8/25/2025.

17) Creating namespace and event hub

The screenshot shows the Microsoft Azure portal interface. The browser address bar displays the URL: `https://portal.azure.com/#@michaelkloudkraft@hotmail.onmicrosoft.com/resource/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resour...`. The page title is "aksevent (aks-eventhub/aksevent)" and it is identified as an "Event Hubs Instance". A notification banner at the top states: "You can start generating test data or inspect data that has already been sent with the new Azure Event Hubs Data Explorer. Click on this message to try the feature!".

The left sidebar contains a navigation menu with the following items: Overview, Access control (IAM), Diagnose and solve problems, Data Explorer, Resource visualizer, Settings, Entities, Features, Automation, and Help. The "Overview" section is currently selected.

The main content area displays the "Essentials" section for the Event Hubs Instance. It includes the following details:

- Resource group: [LabsKraft2027](#)
- Location: East US
- Subscription: [LabsKraft](#)
- Subscription ID: d008c7cc-9795-4292-bf79-74ae52cda63d
- Partition count: 1
- Status: [Active](#)
- Namespaces: [aks-eventhub](#)
- Created: Monday, August 25, 2025 at 20:30:04 GMT+5:30
- Updated: Monday, August 25, 2025 at 20:30:04 GMT+5:30
- Cleanup policy: [Delete](#)

Below the Essentials section, there are two action cards:

- Capture events**: Use Capture to save your events to persistent storage.
- Process data**: Process data instantly with Azure Stream Analytics.

The bottom of the screen shows a Windows taskbar with the date and time: 8:30 PM, 8/25/2025.

18) Generating shared access policies

The screenshot shows the Microsoft Azure portal interface, specifically the "SAS Policy: akspolicy" page for the "aksevent (aks-eventhub/aksevent)" Event Hubs Instance. The browser address bar displays the URL: `https://portal.azure.com/#@michaelkloudkraft@hotmail.onmicrosoft.com/resource/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resour...`.

The left sidebar contains a navigation menu with the following items: Overview, Access control (IAM), Diagnose and solve problems, Data Explorer, Resource visualizer, Settings, Entities, Features, Automation, and Help. The "Settings" section is currently selected, and the "Shared access policies" sub-section is active.

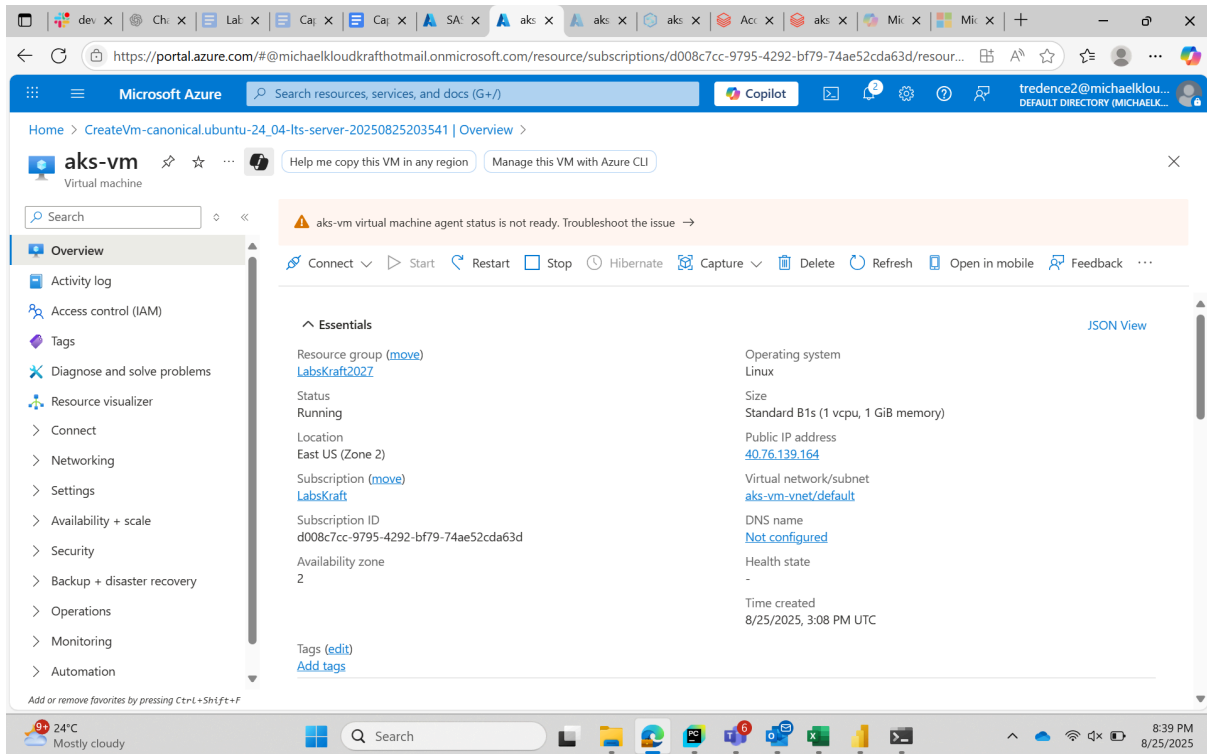
The main content area displays the "SAS Policy: akspolicy" page. It includes the following details:

- Event Hubs access control with Shared Access Signatures [Learn more](#)
- Claims: ☒ Manage, ☒ Send, ☒ Listen
- Primary key: [Redacted]
- Secondary key: [Redacted]
- Primary connection string: [Redacted]
- Secondary connection string: [Redacted]
- SAS policy ARM ID: `/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/labsKraft2027/providers/Microsoft.EventHub/namespaces/aks-eventhub/eventHubs/akspolicy`

At the bottom of the page, there are buttons for "Save", "Discard changes", and a "Give feedback" link.

The bottom of the screen shows a Windows taskbar with the date and time: 8:31 PM, 8/25/2025.

19) Creating a virtual machine to send data to the event hub



20) Generating data using the below python script to send data to the event hub

```
import random
import time
import uuid
from datetime import datetime
from azure.eventhub import EventHubProducerClient, EventData
```

```
# -----
# CONFIGURATION
# -----
EVENT_HUB_CONNECTION_STR =
"Endpoint=sb://hetulproject01eventspace.servicebus.windows.net/;SharedAccessKeyName=
hello;SharedAccessKey=Cqw6MObIR0mIlhVGqOjbbEtH6s6lvgl2+AEhDNW7r4=;EntityPath
=hetulhub"
EVENT_HUB_NAME = "hetulhub"

# -----
# Function to generate one IoT row
# -----
```

```

def generate_row():
    quantity = random.randint(1, 10)
    unit_price = round(random.uniform(50, 500), 2)
    sales_amount = round(quantity * unit_price, 2)
    device_status = random.choice(["Active", "Inactive", "Error"])
    anomaly_flag = 1 if device_status != "Active" else 0
    anomaly_type = random.choice(["Overheat", "LowBattery", "Disconnected"]) if
anomaly_flag else "None"

```

```

    return {
        "OrderID": str(uuid.uuid4()),
        "CustomerID": f"CUST{random.randint(1000,9999)}",
        "Region": random.choice(["North", "South", "East", "West"]),
        "StoreID": random.choice(["Store001", "Store002", "Store003"]),
        "Product": random.choice(["Laptop", "Phone", "Monitor", "Keyboard", "Mouse"]),
        "Quantity": quantity,
        "UnitPrice": unit_price,
        "SalesAmount": sales_amount,
        "OrderTime": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "DeviceType": random.choice(["SensorA", "SensorB", "SensorC"]),
        "Temperature": round(random.uniform(20, 100), 2),
        "DeviceStatus": device_status,
        "AnomalyFlag": anomaly_flag,
        "AnomalyType": anomaly_type
    }

```

```

# -----
# Event Hub Producer
# -----
producer = EventHubProducerClient.from_connection_string(
    conn_str=EVENT_HUB_CONNECTION_STR,
    eventhub_name=EVENT_HUB_NAME
)

```

```

# -----
# Send data every second
# -----
try:
    for _ in range(1000): # Send exactly 1000 rows
        row = generate_row()
        event_data = EventData(str(row)) # convert dict to string
        with producer:
            producer.send_batch([event_data])
        print(f"Sent row: {row}")

```

```

        time.sleep(1) # 1-second interval
except KeyboardInterrupt:
    print("Streaming stopped by user.")
finally:
    producer.close()

```

21) Setting up event hub config in databricks

```

# Event Hub connection parameters

eh_namespace = "aks-eventhub"

eh_name = "aksevent"

eh_sas_key_name = "akspolicy"

eh_sas_key = "PNxlad35DqJlmuKdR3b0SUl45WH7064Lo+AEhPthK/0="

connection_string =
f"Endpoint=sb://aks-eventhub.servicebus.windows.net/;SharedAccessKeyName=aksp
olicy;SharedAccessKey=PNxlad35DqJlmuKdR3b0SUl45WH7064Lo+AEhPthK/0=;EntityPath
=aksevent"

```

```

from pyspark.sql.types import StructType, StructField, StringType,
DoubleType, TimestampType, IntegerType

from pyspark.sql import functions as F

# Define schema for Event Hub messages

event_schema = StructType([

    StructField("OrderID", StringType(), True),

    StructField("CustomerID", StringType(), True),

    StructField("Region", StringType(), True),

    StructField("StoreID", StringType(), True),

```

```

    StructField("Product", StringType(), True),

    StructField("Quantity", IntegerType(), True),

    StructField("UnitPrice", DoubleType(), True),

    StructField("SalesAmount", DoubleType(), True),

    StructField("OrderTime", StringType(), True),      # can cast to
Timestamp later in Silver

    StructField("DeviceType", StringType(), True),

    StructField("Temperature", DoubleType(), True),

    StructField("DeviceStatus", StringType(), True),

    StructField("AnomalyFlag", IntegerType(), True),

    StructField("AnomalyType", StringType(), True)

])

# Read streaming data from Event Hub

stream_df = (spark.readStream

    .format("eventhubs")

    .option("eventhubs.connectionString", connection_string)

    .load()

)

# Event Hub messages are in 'body' column as binary, decode them

decoded_df = (stream_df

    .withColumn("body_str", F.col("body").cast("string"))

    .select(F.from_json("body_str", event_schema).alias("data"))

    .select("data.*")

```

```

)

# Write stream to Bronze Delta table

bronze_stream = (decoded_df.writeStream

    .format("delta")

    .outputMode("append")

    .option("checkpointLocation", "/mnt/delta/events_bronze/_checkpoints/")

    .table("aks_proj1.bronze.events_bronze") # new bronze table for
streaming

)

```

We get an error because we don't have admin access.

The screenshot shows a Databricks notebook interface. The left sidebar contains navigation options like Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie, Alerts, Query History, and SQL Warehouses. The main area displays a Python script with the following code:

```

24 .format("eventhubs")
25 .option("eventhubs.connectionString", connection_string)
26 .load()
27 )
28
29 # Event Hub messages are in 'body' column as binary, decode them
30 decoded_df = (stream_df
31     .withColumn("body_str", F.col("body").cast("string"))
32     .select(F.from_json("body_str", event_schema).alias("data"))
33     .select("data.*")
34 )
35
36 # Write stream to Bronze Delta table
37 bronze_stream = (decoded_df.writeStream
38     .format("delta")
39     .outputMode("append")
40     .option("checkpointLocation", "/mnt/delta/events_bronze/_checkpoints/")
41     .table("aks_proj1.bronze.events_bronze") # new bronze table for streaming
42 )

```

Below the code, an error message is displayed:

```

> [UNSUPPORTED_STREAMING_SOURCE_PERMISSION_ENFORCED] Data source eventhubs is not supported as a streaming source on a
shared cluster. SQLSTATE: 0A000

```

The error message is highlighted in red. Below the error, there are buttons for "Diagnose error" and "Debug". The bottom status bar shows the temperature as 24°C, the time as 8:54 PM, and the date as 8/25/2025.